A

Project Report

On

❝File Dropper❞

Submitted to

THE SAVITRIBAI PHULE PUNE UNIVERSITY

In Partial fulfilment

Of

BACHELOR OF BUSINSS ADMINISTRATION

(COMPUTER APPLICATION)

(SEM V)

Submitted By

Miss. Amanprit Kaur Gedu
Miss. Siddhi Ghodke

Under The Guidance Of

Asst. Prof. Fatima Tamboli

Through

SARHAD COLLEGE ARTS , COMMERCE & SCIENCE KATRAJ

PUNE-411046

UNIVERSITY OF PUNE 2025

SARHAD COLLEGE OF ARTS ,COMMERCE & SCIENCE

Department of BBA(CA)

Academic year

2025-26

Project Report

On

“File Dropper”

Guided By

Asst. Prof. Fatima Tamboli

- Title of project:

  " File Dropper"

- Submitted By:

  - Team Member Name: Amanprit Kaur Gedu
Siddhi Ghodke

  - Class: TY BBA(CA)

  - Roll No: 233005
233029

(Minority College)
Sarhad College of Arts, Cömmerce & Science
[Affiliated to Savitribai Phule Pune University (ID No.PUIPNIACS130012008) and Recognised by Government of Maharashtra]
Sarhad Bhavan, Pune.KashmirMaitrUe Chowk, S.No. (New), Behind Bharati Vidyapeeth, Katraj, Pune46.
Ph.:+91 20 24368621 | email: sarhadcollege@gmail.com I website: www.sarhadcollege.education
Unipune ID : CAAP013640 1 A.I.S.H.E. Code : '6/4,79,80,81C - 41270 | Exam No. 0847

Project Certificate

This is to certify that Miss/Mr.Amanprit Kaur Gedu
Satisfactory completed the Project as required by Savitribai Phule Pune University for T.Y.BBA CA Semester-VI, in the academic year 2025-2026.

Project Title: File Dropper

Project Guide HOD Principal

Internal Examine External Examiner

H.O. : Gopal Krishna Gokhale Prabodhini, 6th Floor, Sarhad Public School & Jr. College, S. No. 61213B, Dhankawadi, Pune 411043. Ph. : +91-8485871551

Acknowledgement

We would like to express our wholehearted thanks to all those who directly and indirectly helped me during the project. Honestly speaking, it would have been impossible to complete this project without valuable guidance from various sources. We'd like to thank Mrs. Fatima Tamboli ma'am for their constant encouragement and support throughout the project. A project is an attempt by every student to put his skills to use and produce something he desired. It is the first time that he comes to know the real difficulties of developing the system, which will be tested and implemented. We had a similar dream that has now come true. On the successful completion of our project work, we take this opportunity to thank every person associated with it.

Last but not least, a special thanks to all our friends for their great support and guidance throughout the project.

Thanking You

Index

ABSTRACT

The File Dropper – File Sharing Web Application is a modern, secure, and scalable platform designed to simplify file uploading and sharing through dynamically generated links. The system enables users to upload files, configure sharing settings such as password protection, expiry time, and download limits, and share files efficiently across different devices.

The application is developed using a full-stack architecture. The frontend is built using **React**, along with **HTML, CSS, JavaScript, and Bootstrap**, providing a responsive and interactive user interface across multiple platforms. The backend is implemented using **Python frameworks such as Flask/FastAPI**, which handle API requests, file processing, authentication, and core business logic.

The system uses **MongoDB**, a NoSQL database, to store file metadata, user information, and sharing configurations. MongoDB offers flexibility and scalability, making it suitable for handling dynamic and semi-structured data efficiently.

The platform incorporates essential security features including password-protected access, controlled download limits, and configurable file expiry, ensuring secure and controlled file sharing. Additionally, the system provides fast performance and efficient file handling through optimized backend processing.

Overall, the File Dropper application delivers a reliable and user-friendly solution for modern file sharing needs. This project demonstrates practical implementation of full-stack web development, REST API integration, and NoSQL database management, making it a scalable and real-world applicable system.

1. Introduction

1.1 Introduction to System

The **File Dropper – File Sharing Web Application** is a modern web-based system designed to provide users with a fast, secure, and efficient way to upload and share files over the internet. The application allows users to upload files, generate shareable links, and control access through features such as password protection, expiry time, and download limits.

The system is built using a full-stack architecture, where the frontend is developed using **React along with HTML, CSS, JavaScript, and Bootstrap**, providing a responsive and interactive user interface. The backend is implemented using **Python frameworks such as Flask/FastAPI**, which handle API requests, file processing, authentication, and business logic. The system uses **MongoDB**, a NoSQL database, to store user data, file metadata, and sharing configurations.

The primary goal of the system is to simplify file sharing while ensuring data security and efficient performance. The platform is designed to be user-friendly, enabling users to share files instantly with minimal effort.

1.2 Existing System

Traditional file sharing methods such as email attachments and messaging platforms have several limitations:

- File size restrictions
- Lack of secure access control
- No option for expiry or download limits
- Difficulty in managing shared files
- Limited tracking and control

These limitations reduce efficiency and create security concerns for users.

1.3 Proposed System

The proposed system overcomes the limitations of traditional file sharing methods by providing a centralized and secure platform for file upload and sharing.

Key features of the proposed system include:

- Easy file upload with drag-and-drop functionality
- Generation of secure shareable links
- Password-protected file access
- Configurable expiry time and download limits
- File management dashboard
- Fast and responsive user interface

The system ensures better performance, improved security, and enhanced user experience.

1.4 Objectives of the System

The main objectives of the system are:

- To develop a secure file sharing platform
- To provide fast and efficient file upload functionality
- To enable controlled file access through security features
- To create a user-friendly and responsive interface
- To ensure data integrity and reliability

1.5 Scope of the System

The scope of the File Dropper system includes:

- Uploading and sharing files via links
- Managing file access through password and expiry settings
- Viewing and managing uploaded files
- Ensuring secure data handling and storage using MongoDB

The system can be further extended in the future with advanced features such as user authentication systems, cloud storage integration, and mobile application support.

2. SYSTEM ANALYSIS

2.1 Feasibility Study

The feasibility study evaluates whether the proposed File Dropper system is practical and beneficial in real-world implementation.

a) **Technical Feasibility**

The system is technically feasible as it is built using modern and widely used technologies:

- **Frontend:** React, HTML, CSS, JavaScript, Bootstrap
- **Backend:** Python (Flask / FastAPI)
- **Database:** MongoDB

These technologies are open-source, well-documented, and suitable for developing scalable web applications. The development tools such as VS Code and Postman further simplify implementation and testing.

b) **Operational Feasibility**

The system is user-friendly and easy to operate. Users can upload files, set sharing options, and generate links with minimal technical knowledge. The interface is responsive and accessible across multiple devices such as desktops, tablets, and mobile phones.

c) **Economic Feasibility**

The project is cost-effective as it uses open-source technologies and requires minimal infrastructure. No expensive hardware or software licenses are needed, making it suitable for academic and small-scale deployment.

2.2 System Requirements

The system adopts a modern, distributed architecture, minimizing the need for specific client-side hardware while relying on enterprise-grade cloud computing for the backend.

A. **Software Requirements**
   - **Operating System:** Windows, macOS, or Linux (for development and deployment).
   - **Frontend:** React, HTML, CSS, JavaScript, Bootstrap

     Backend Language: Python 3+ (utilizing Flask or Django for API construction).
   - **Database Management System:** MongoDB (NoSQL Document Database).
   - **Tools:** VS Code, Postman, Web Browser

B. **Hardware Requirements (Minimum for Development/Testing)**
   - **Minimum 4 GB RAM**
   - **Processor: Intel i3 or higher**
   - **Hard Disk: 20 GB free space**
   - **Internet connection**

2.3 Requirement Analysis

Functional Requirements

- Upload files
- Generate shareable links
- Set password protection
- Configure expiry time
- Set download limits
- View uploaded files
- Manage file access

Non-Functional Requirements

- **Performance:** Fast file upload and download
- **Security:** Password protection and input validation
- **Scalability:** Ability to handle multiple users and files
- **Usability:** Simple and intuitive interface
- **Reliability:** Stable system with minimal errors

2.4 System Constraints

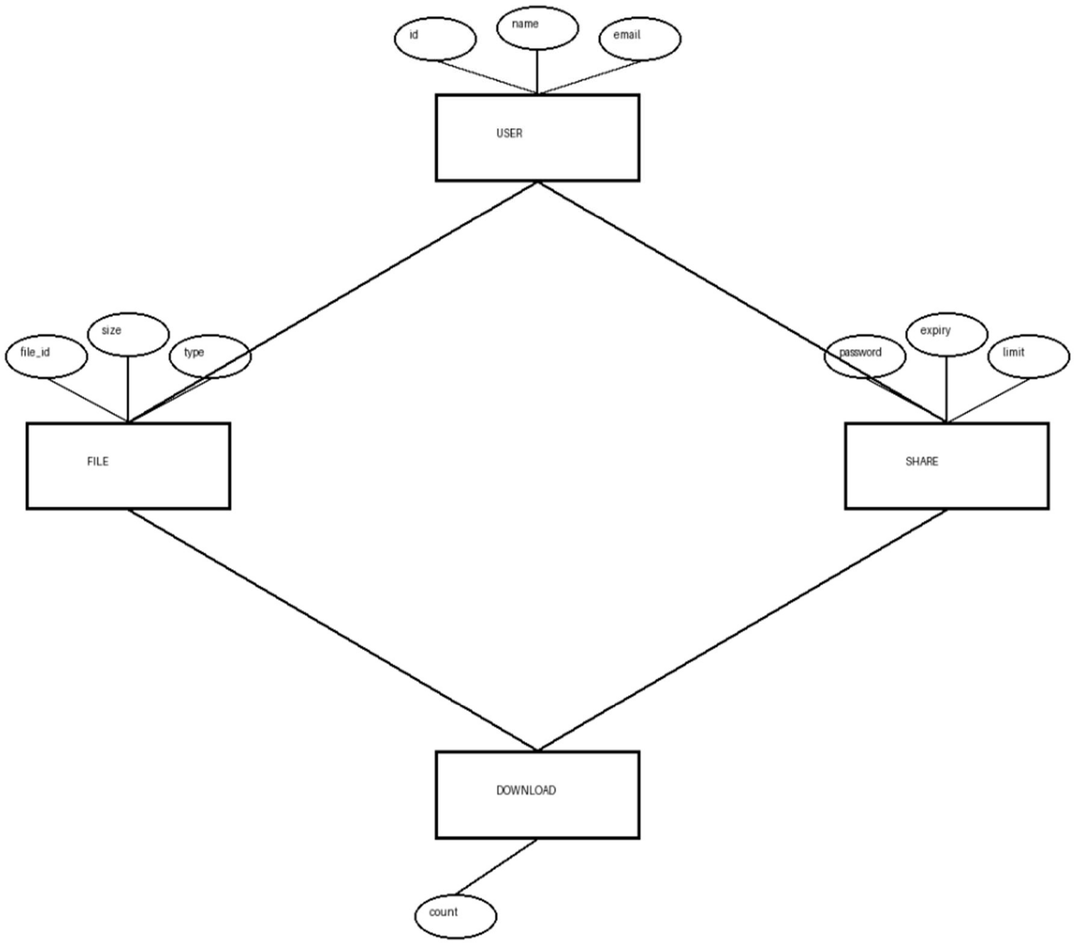
❖ File size limitations depending on server capacity
❖ Internet connectivity required
❖ Storage limitations for uploaded files
❖ Performance depends on server resources

2.5 Assumptions

❖ Users have basic knowledge of web applications
❖ Stable internet connection is available
❖ The system will be used for general file sharing purposes
❖ Users follow security guidelines while sharing files

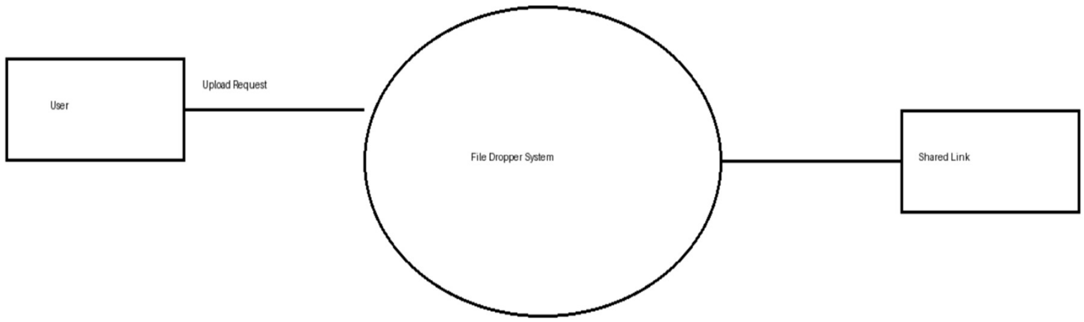
3. System Design

3.1 Design Constraints

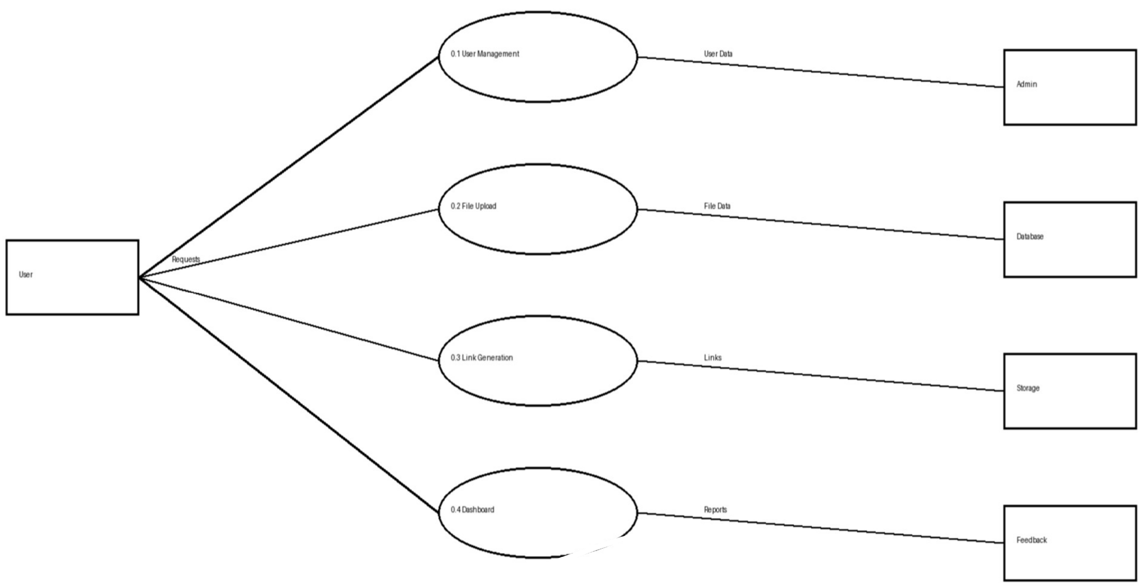


3.2 System Model

1. Data Flow Diagram (DFD) - Zeroth Level (0.0 level)

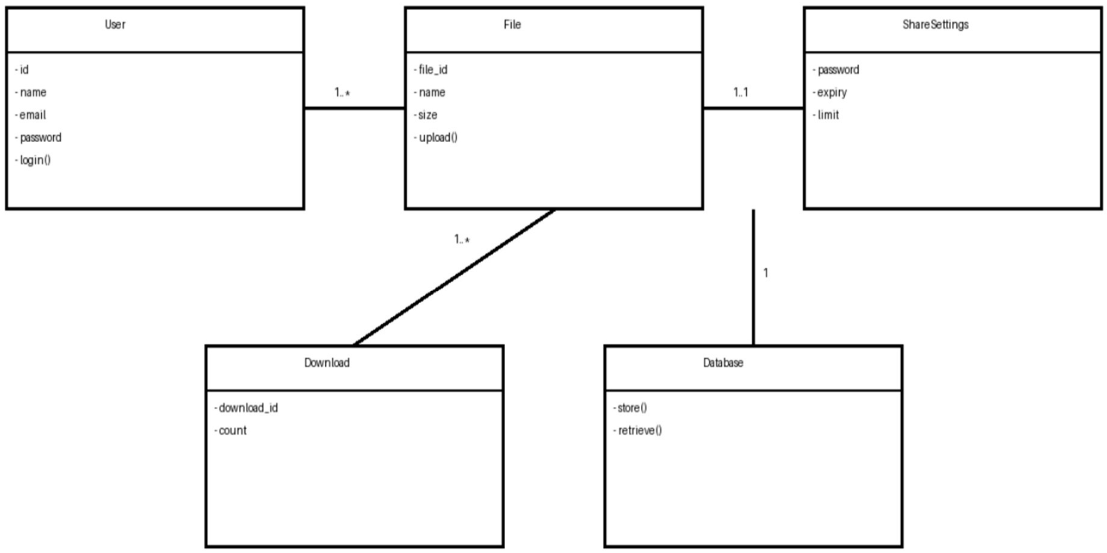


2. DataFlow Diagram (DFD) - First Level (0.1 Level)

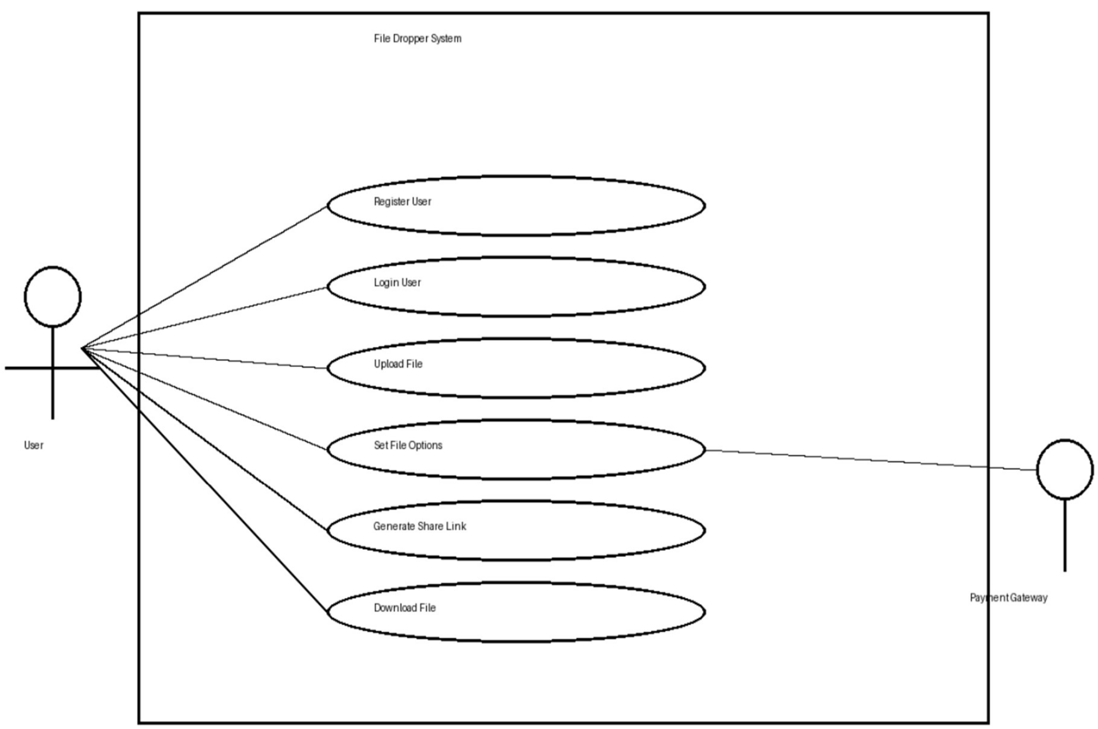


3.3 Data Modeling

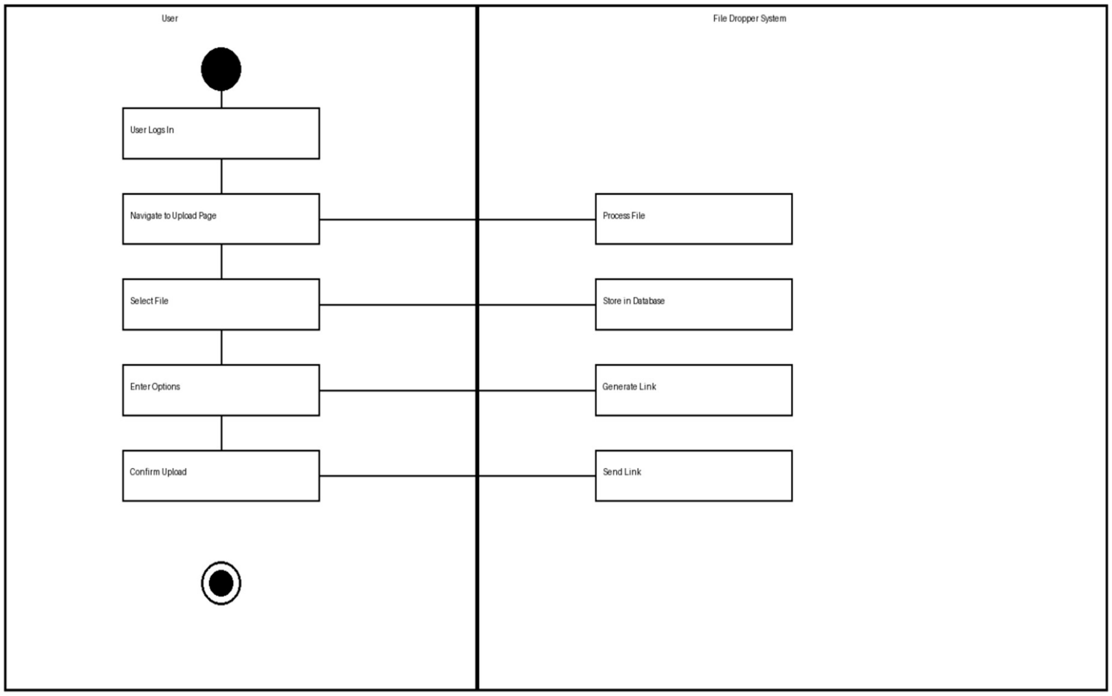
1. Class Diagram



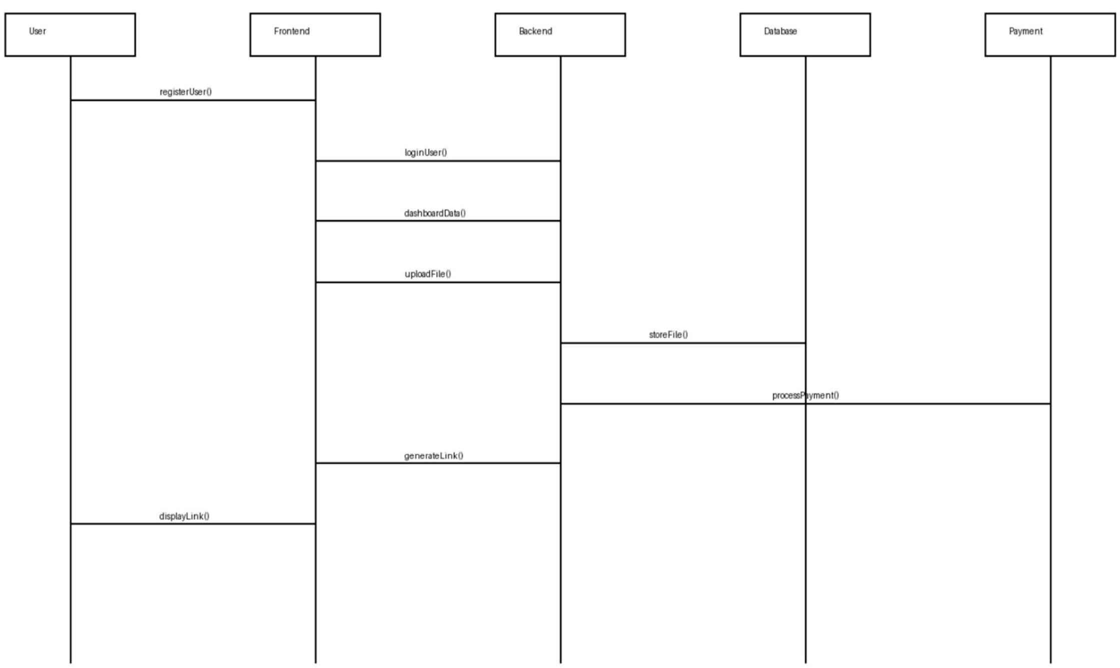
2. Use Case Diagram



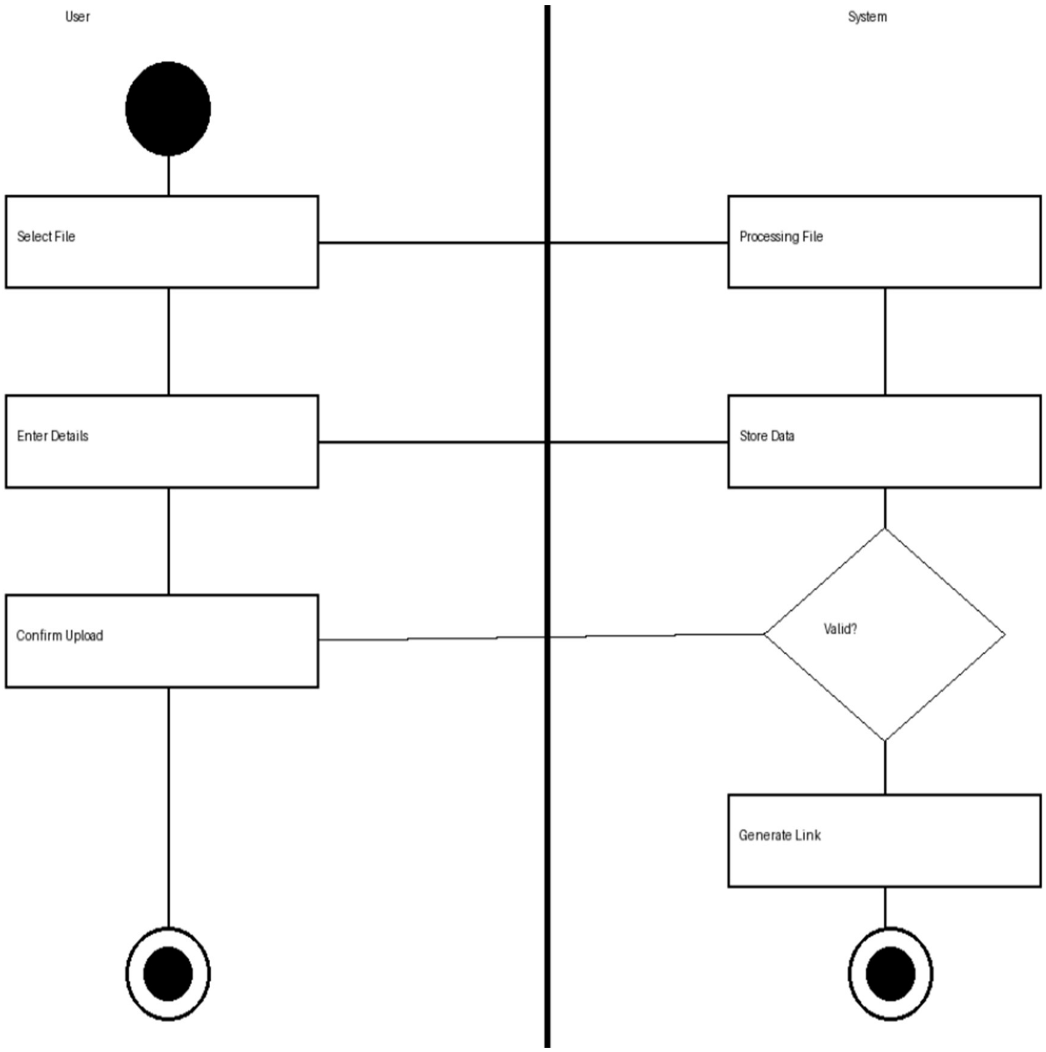
3. Activity Diagram



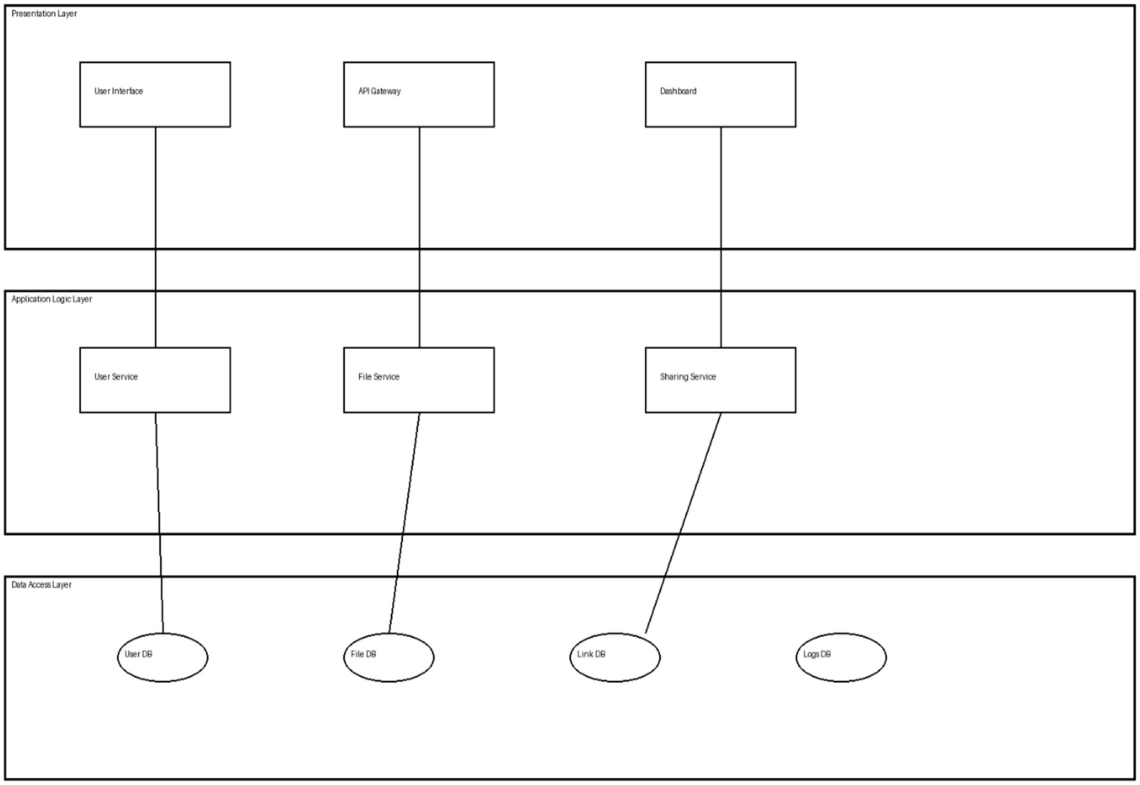
4. Sequence Diagram



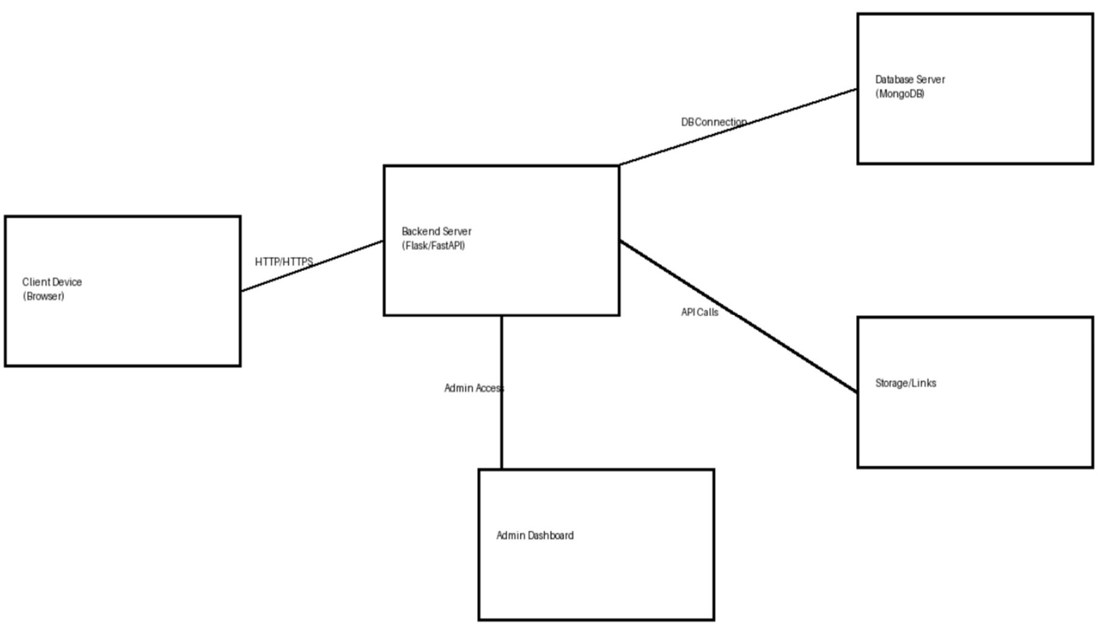
5. State Chart Diagram



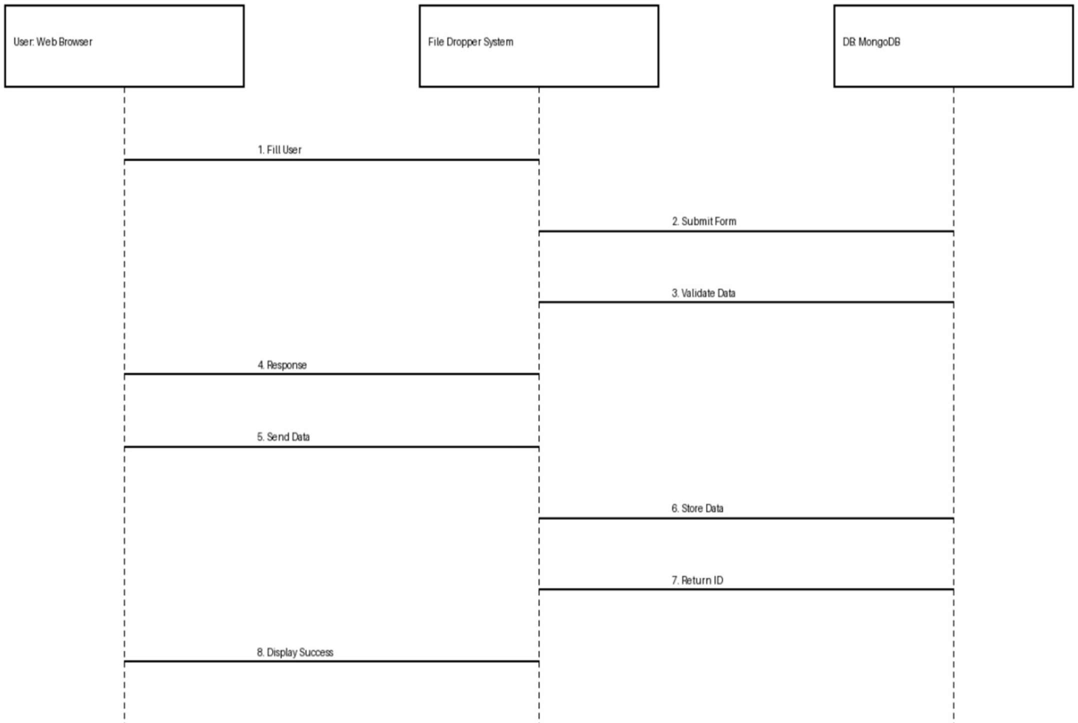
6. Component Diagram



7. Deployment Diagram



8. Collaboration Diagram



4. Data Dictionary

Table: users

Fieldname	Data Type	Key	Description
id	Int	Primary	Unique identifier for each user
full_name	Text	–	User's full name
email	Text	Unique	User's email address (used for login)
password_hash	Text	–	Hashed password for secure login
grad_year	Int	–	Year of graduation
is_admin	Boolean	–	Indicates whether user is admin
created_at	DateTime	–	Date and time of account creation

Table: files

Fieldname	DataType	Key	Description
file_id	Int	Primary	Unique identifier for file
user_id	Int	Foreign	Reference to user who uploaded file
file_name	Text	–	Name of uploaded file
file_size	Int	–	Size of file in MB
file_type	Text	–	Type/format of file
upload_date	DateTime	–	Date of file upload

Table: share_links

Fieldname	DataType	Key	Description
link_id	Int	Primary	Unique identifier for share link
file_id	Int	Foreign	Reference to uploaded file
share_url	Text	–	Generated shareable link
password	Text	–	Optional password protection
expiry_date	DateTime	–	Link expiration date
download_limit	Int	–	Maximum number of downloads allowed

Table: downloads

Fieldname	DataType	Key	Description
download_id	Int	Primary	Unique download record
file_id	Int	Foreign	Reference to file
user_id	Int	Foreign	User who downloaded file
download_time	DateTime	–	Timestamp of download

OUTPUTS AND REPORT TESTING

Test Cases

1. Test Cases for Home Page (Public Access)

Sr No	Test Case ID	Feature	Description	Steps to Execute	Expected Result
1	TC-H-001	Page Load	Verify home page loads successfully	Open website URL	Page loads without errors within 3–5 seconds
2	TC-H-002	Responsiveness	Check responsiveness on devices	Open site on mobile, tablet, desktop	Layout adjusts correctly without UI breaking
3	TC-H-003	Navigation	Verify navigation menu works	Click menu links	All pages open correctly

---

2. Test Cases for User Authentication

Sr No	Test Case ID	Feature	Description	Steps to Execute	Expected Result
1	TC-A-001	Registration	Verify user registration	Fill form and submit	User registered successfully
2	TC-A-002	Login	Verify login with valid credentials	Enter correct email & password	User logged in successfully
3	TC-A-003	Invalid Login	Check invalid login	Enter wrong credentials	Error message displayed

---

3. Test Cases for File Upload

Sr No	Test Case ID	Feature	Description	Steps to Execute	Expected Result
1	TC-F-001	File Upload	Upload a valid file	Select file and click upload	File uploaded successfully
2	TC-F-002	File Size Limit	Upload large file	Upload file > allowed size	Error message shown
3	TC-F-003	File Type	Upload unsupported format	Upload invalid file type	Upload rejected

---

4. Test Cases for Share Link Generation

Sr No	Test Case ID	Feature	Description	Steps to Execute	Expected Result
1	TC-S-001	Generate Link	Create share link	Upload file and generate link	Unique link generated
2	TC-S-002	Expiry Setting	Check expiry feature	Set expiry date	Link expires after given time
3	TC-S-003	Password	Verify password protection	Set password on file	File accessible only with password

---

5. Test Cases for File Download

Sr No	Test Case ID	Feature	Description	Steps to Execute	Expected Result
1	TC-D-001	Download File	Download shared file	Open link and download	File downloads successfully
2	TC-D-002	Expired Link	Check expired link	Open expired link	Access denied message
3	TC-D-003	Download Limit	Check download limit	Download multiple times	Stops after limit reached

White Box Testing

White Box Testing evaluates the internal code structure, logic, and database operations of the system. It focuses on analyzing code paths, algorithms, and internal workflows to ensure correctness, efficiency, and security.

In the File Dropper System, white box testing is applied to critical backend functionalities such as authentication handling, file upload processing, secure link generation, and database operations.

Key Areas: Authentication Handlers, File Upload Logic, Link Generation Logic, and Database Queries.

White Box Test Cases

Test Case	Description	Expected Outcome
Authentication Logic	Analyze code paths for login/registration to ensure secure password hashing and session handling.	Code follows secure hashing (bcrypt/sha); no plaintext passwords stored; sessions handled securely.
File Upload Logic	Trace execution of file upload process including validation and storage.	File is validated, stored correctly, and no unauthorized files are accepted.
Link Generation Logic	Verify unique share link generation and expiry logic implementation.	Unique links generated; expiry and limits work correctly without conflicts.
Download Flow Logic	Analyze file download process and access control checks.	Only authorized users/downloads allowed; expired links are blocked.
Database Operations	Validate CRUD operations and data consistency.	Data stored and retrieved correctly; no redundancy or inconsistency.
Input Validation	Check backend validation for user inputs.	System prevents invalid inputs, injection attacks, and malicious data.

Black Box Testing

Black Box Testing evaluates the functional behavior of the system without any knowledge of the internal code structure. It ensures that the system meets user requirements and performs correctly for all end-to-end operations.

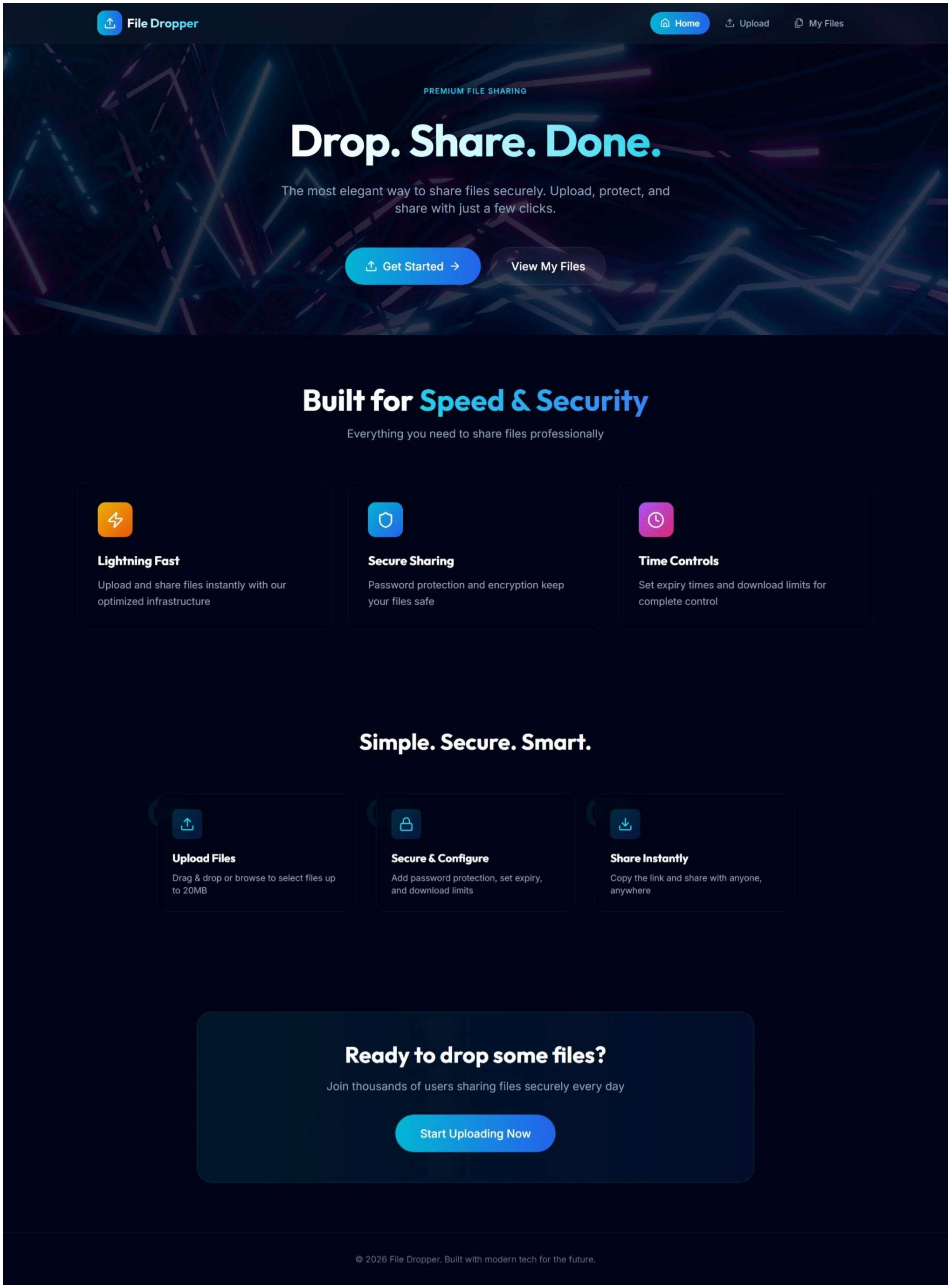
In the File Dropper System, black box testing focuses on user interactions such as file upload, sharing, downloading, and authentication processes.

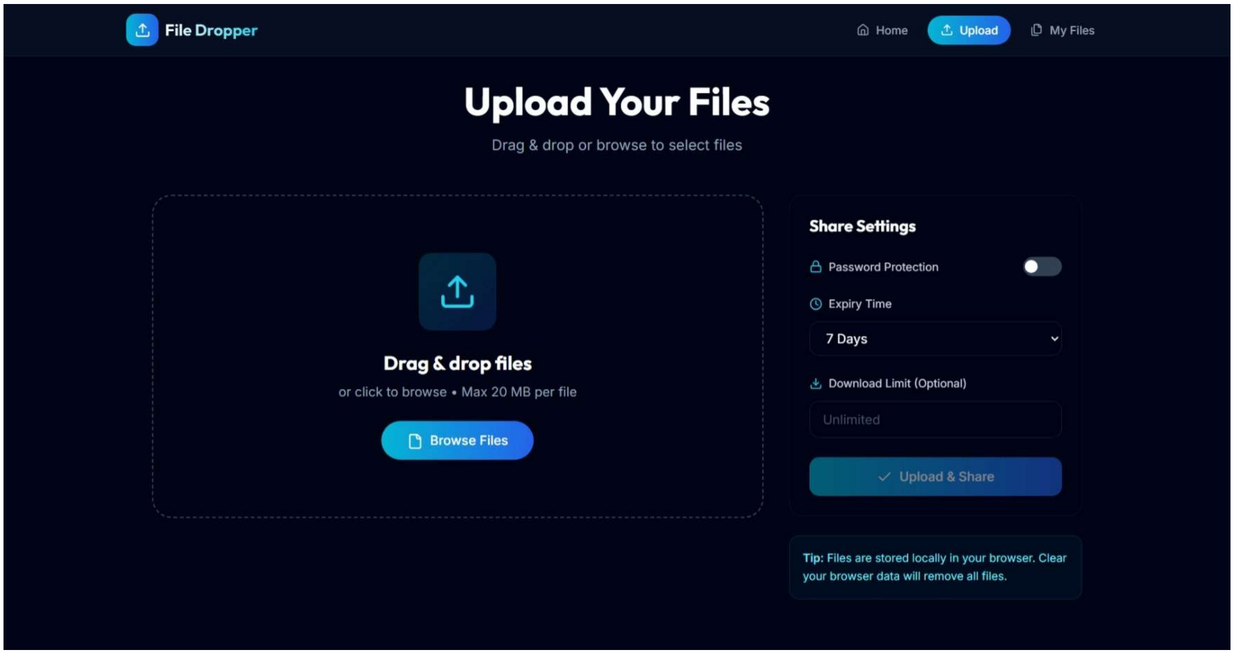
Focus: End-to-end user flows like registration, login, file upload, link generation, and file download.

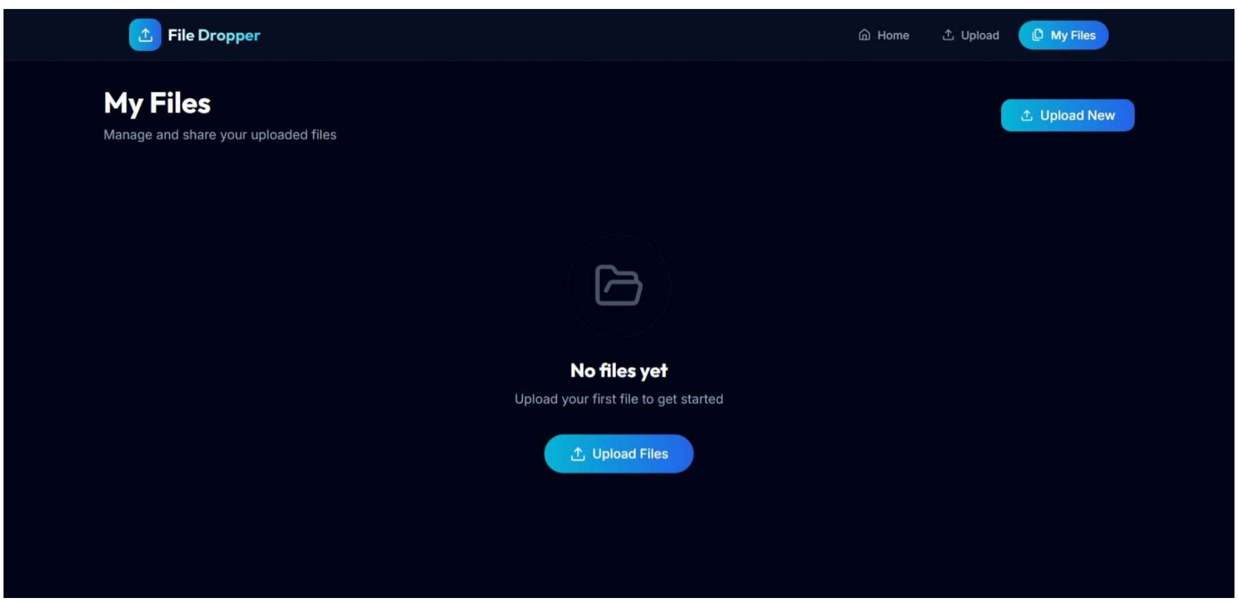
Black Box Test Cases

Test Case	Description	Expected Outcome
End-to-End File Upload Flow	Verify complete upload process: User selects file → Uploads → Receives share link.	File is uploaded successfully; unique share link is generated and displayed.
User Registration	Verify user can register with valid details	User account created successfully; confirmation shown
User Login	Verify login with correct credentials	User is logged in and redirected to dashboard
Invalid Login	Check login with incorrect credentials	Error message displayed; access denied
File Download	Verify file can be downloaded using valid link	File downloads successfully
Expired Link Access	Check behavior of expired links	Access denied with appropriate message
Password Protected File	Verify access to password-protected file	File accessible only after correct password entry
Download Limit	Verify download limit functionality	Download stops after limit is reached

SCREENSHOT AND REPORT




File Dropper
Home
Upload
My Files
Upload Your Files
Drag & drop or browse to select files
Drag & drop files
or click to browse • Max 20 MB per file
Browse Files
Share Settings
Password Protection
Expiry Time
7 Days
Download Limit (Optional)
Unlimited
Upload & Share
Tip: Files are stored locally in your browser. Clear your browser data will remove all files.


File Dropper
Home
Upload
My Files
My Files
Manage and share your uploaded files
Upload New
No files yet
Upload your first file to get started
Upload Files

5. PROJECT FUTURE SCOPE

The File Dropper System developed in this project provides a secure and efficient platform for uploading and sharing files. However, there are several opportunities for further enhancement and expansion to improve functionality, performance, and user experience.

In the future, the system can be enhanced by integrating **cloud storage services** such as AWS S3, Google Drive, or Azure Storage to enable scalable and reliable file storage. This will allow users to upload larger files and ensure better availability and backup mechanisms.

Another important improvement is the implementation of **user authentication and role-based access control**, where users can have different roles such as admin and normal user. This will improve security and allow better management of uploaded content.

The system can also be upgraded by adding **real-time file tracking and analytics**, where users can monitor the number of downloads, link usage, and access statistics. This feature will provide better insights into file sharing activity.

To enhance security, future versions can include **advanced encryption techniques**, secure token-based sharing, and protection against threats such as unauthorized access, malware uploads, and brute-force attacks.

Additionally, the platform can be extended to support **mobile applications (Android/iOS)**, allowing users to upload and access files on the go. Integration with notifications (email/SMS) can also be implemented to inform users about file downloads and link expiry.

Other enhancements may include:

- **Drag-and-drop multi-file upload**
- **File preview (images, PDFs, videos)**
- **Version control for files**
- **User dashboard with activity history**
- **Faster upload using chunking for large files**

In conclusion, the system has strong potential to evolve into a full-featured, enterprise-level file sharing platform with improved scalability, security, and usability.

6. CONCLUSION AND RECOMMENDATIONS

The File Dropper System successfully achieves its objective of providing a simple, secure, and efficient platform for uploading and sharing files. The system is developed using modern technologies including React for the frontend, Python (Flask/FastAPI) for backend processing, and MongoDB for data storage, ensuring a scalable and responsive application.

The project implements essential features such as file upload, secure link generation, download management, and user interaction through an intuitive interface. Proper validation mechanisms and structured data handling ensure data integrity and reliability. Additionally, testing methodologies such as white box and black box testing were applied to verify system performance, functionality, and security. The results confirm that the system meets its intended requirements and performs effectively under standard conditions.

Despite its successful implementation, there is significant scope for further improvement. Future enhancements can greatly improve the system's performance, usability, and scalability. Integrating cloud storage services such as AWS S3 or Google Cloud Storage can provide better file management and support for large-scale data. Advanced security features like encryption, token-based authentication (JWT), and protection against vulnerabilities can further strengthen system security.

The system can also be enhanced by introducing role-based access control, allowing different levels of user permissions such as admin and regular users. Performance can be improved through file chunking and optimization techniques, especially for handling large files. Additionally, features such as file preview, version control, and real-time analytics can provide a richer user experience.

Furthermore, developing mobile applications and integrating notification systems (email/SMS) can improve accessibility and user engagement. With these enhancements, the File Dropper System has strong potential to evolve into a comprehensive, enterprise-level file sharing solution.

7. BIBLIOGRAPHY AND REFERENCES

The development of the File Dropper System involved the use of various books, online resources, official documentation, and tools. These references helped in understanding the concepts of web development, backend processing, database management, and system design.

1. **Duckett, J. (2014). *HTML and CSS: Design and Build Websites*. Wiley Publications.**
2. **Duckett, J. (2014). *JavaScript and jQuery: Interactive Front-End Web Development*. Wiley Publications.**
3. **Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.**
4. **Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.**
5. **Freeman, E., & Robson, E. (2004). *Head First Design Patterns*. O'Reilly Media.**
6. **React Official Documentation – https://react.dev**
7. **Bootstrap Documentation – https://getbootstrap.com**
8. **Flask Documentation – https://flask.palletsprojects.com**
9. **FastAPI Documentation – https://fastapi.tiangolo.com**
10. **MongoDB Documentation – https://www.mongodb.com/docs**
11. **MDN Web Docs (HTML, CSS, JavaScript) – https://developer.mozilla.org**
12. **Postman API Testing Tool – https://www.postman.com**
13. **Visual Studio Code Documentation – https://code.visualstudio.com/docs**